HetLitmus

Generation of CPU-GPU Heterogeneous Litmus Tests for Real Hardware

Nguyen Le
Advisor: Dr. Anatole Lefort
Chair of Computer Systems
https://dse.in.tum.de/



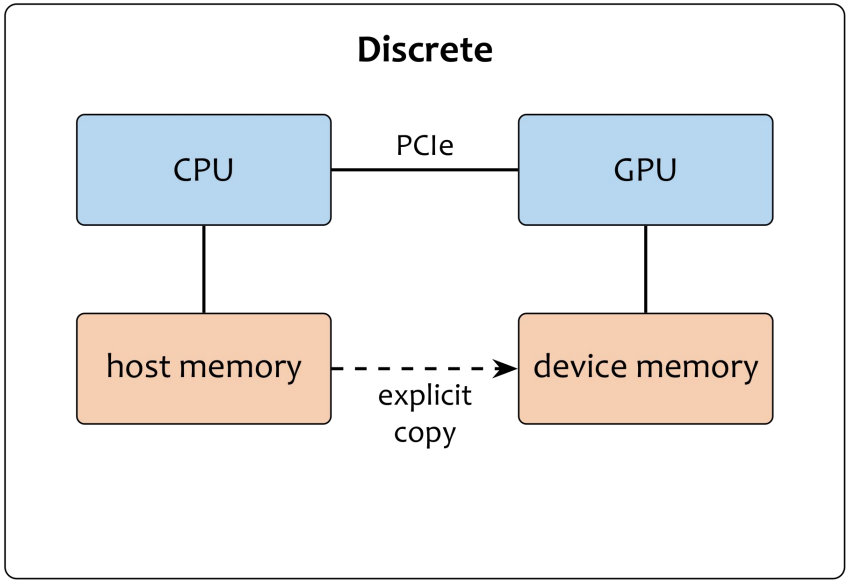
15.06.2026 – 15.10.2026

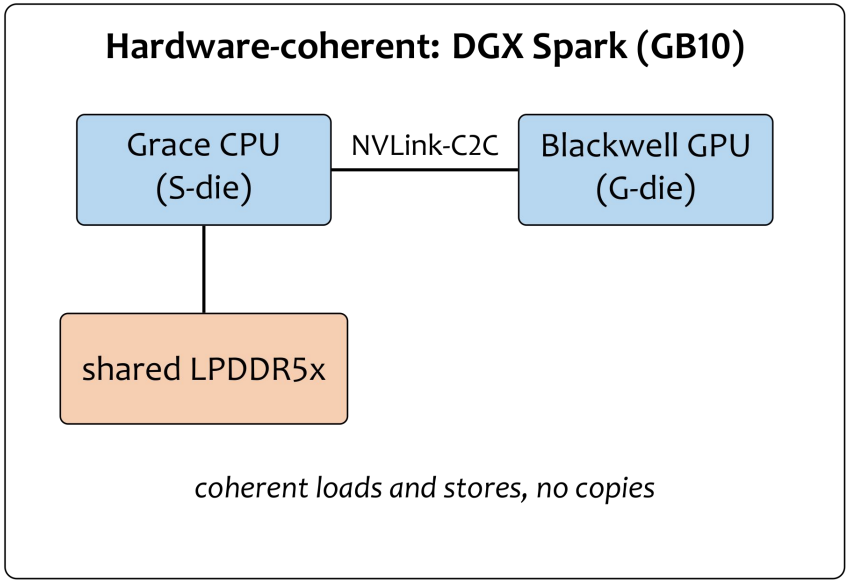
Motivation

- Industry is rolling out systems with CPU and GPU sharing memory
  - NVIDIA GH200 Grace Hopper, AMD Instinct MI300A
- Each device's memory model only answers for itself
- Compound Memory Models (PLDI '23) proposes an answer (only simulation)
- Litmus testing
  - established approach
  - empirical result on real hardware

Background

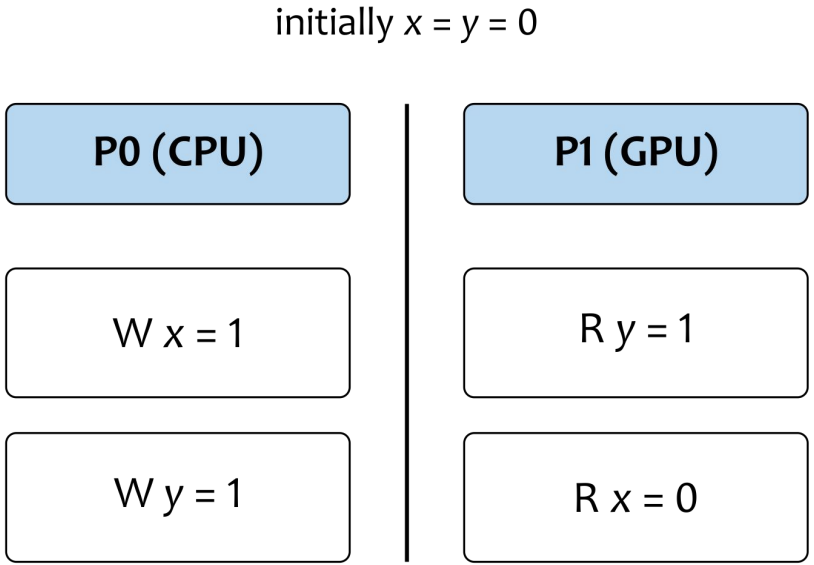
CPU–GPU memory organisation





Background

- Without memory consistency guarantee
  - HW may not run code in the order it's written
  - hidden bug (flag seen, data still old)
- Litmus test
  - run millions of times under stress
  - each shape checks one specific weak behaviour (e.g., MP, SB)
  - one observation is enough to show



weak outcome: P1 sees $y = 1$ but $x = 0$

Example: Message Passing (MP)

Research gap

- single-device litmus testing is mature (herdtools7, GPUHarbor)
- Hand-written
  - ~200 CUDA lines per shape (hetero-cuda-litmus)[1]
  - 1960 variants run on a GH200, but only MP (ISMM '26)[2]
  - ~10-line execution graph + ~20-line JSON config per test (Srivastava)[3]
- GPU backends
  - Mostly CUDA, no HIP
  - Vulkan: built, but didn't work on tested HW (Srivastava)[3]
- No vendor-neutral GPU litmus layer with orders and scopes
  - specify a test once → run across vendors/ISAs
- No interconnect stress

[1] S. Bagchi, hetero-cuda-litmus. https://github.com/sohambagchi/hetero-cuda-litmus
[2] S. Bagchi et al., "Consistency and Coherence of the NVIDIA Grace-Hopper Superchip", ISMM '26. https://doi.org/10.1145/3814942.3816134
[3] S. Srivastava, Testing Memory Models of Heterogeneous CPU-GPU Systems, MSc thesis, UC Santa Cruz, 2024. https://escholarship.org/uc/item/5hm7q1jv

Research gap

- single-device litmus testing is mature (herdtools7, GPUHarbor)
- Hand-written
  - ~200 CUDA lines per shape (hetero-cuda-litmus)[1]
  - 1960 variants run on a GH200, but only MP (ISMM '26)[2]
  - [illegible]
- GPU
  - [illegible]
  - [illegible]
- No vendor-neutral GPU litmus layer with orders and scopes
  - specify a test once → run across vendors/ISAs
- No interconnect stress

How can heterogeneous CPU–GPU litmus tests be systematically enumerated, specified once, and translated faithfully for different CPU and GPU vendors?

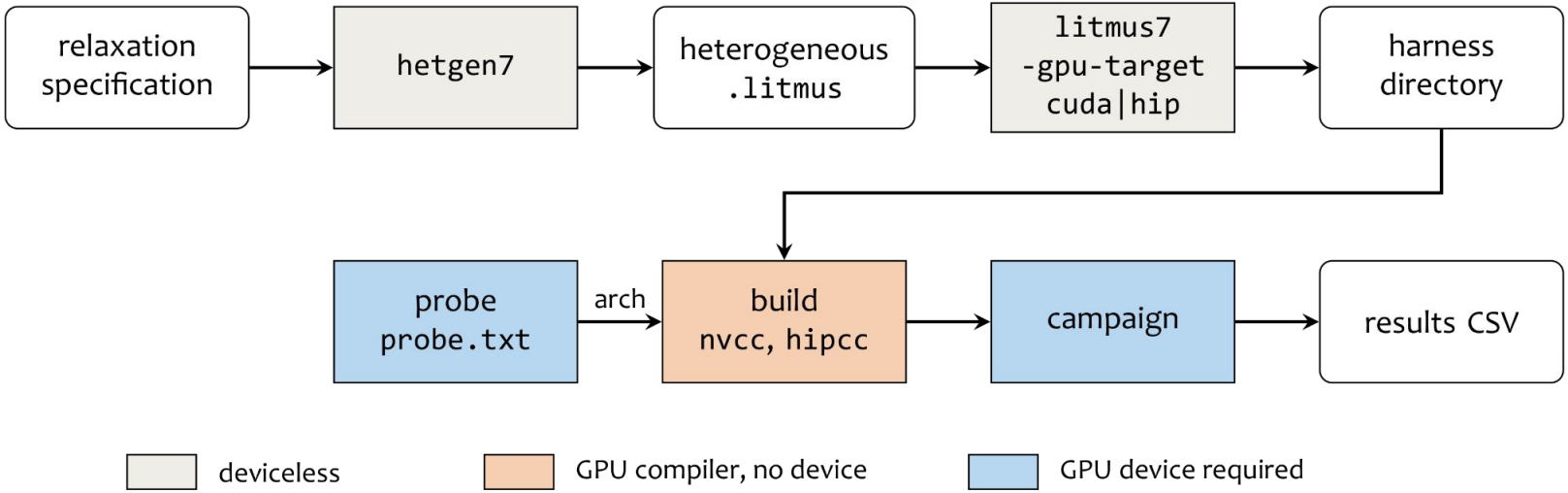
[1] S. Bagchi, hetero-cuda-litmus. https://github.com/sohambagchi/hetero-cuda-litmus
[2] S. Bagchi et al., "Consistency and Coherence of the NVIDIA Grace-Hopper Superchip", ISMM '26. https://doi.org/10.1145/3814942.3816134
[3] S. Srivastava, Testing Memory Models of Heterogeneous CPU-GPU Systems, MSc thesis, UC Santa Cruz, 2024. https://escholarship.org/uc/item/5hm7q1jv

Design goals

- Systematic enumeration
  - across the axes: test families, memory orders, scopes, device cuts
- Vendor independence
  - GPU side written once in a shared vocabulary
  - One code emitter per vendor
- Increase the chance of observation
  - CPU and GPU race every iteration
  - Stress and noise on both devices
- Faithful
  - Compiled code keeps exactly test's orders, scopes and operations

HetLitmus

herdtools7 extension: generate CPU-GPU litmus tests, run on real hardware,
GPU side is translated to CUDA (NVIDIA) or HIP (AMD)



Contribution: Vendor-neutral GPU litmus layer

gpu.bell

```
enum memory_order = 'relaxed || 'acquire || 'release || 'acqrel || 'sc

enum scopes = 'cta || 'gpu || 'sys

let narrower(s) = match s with
  || 'sys -> 'gpu
  || 'gpu -> 'cta
end

let wider(s) = match s with
  || 'gpu -> 'sys
  || 'cta -> 'gpu
end

instructions R[{'relaxed,'acquire,'sc}, scopes]
instructions W[{'relaxed,'release,'sc}, scopes]
instructions F[{'acquire,'release,'acqrel,'sc}, scopes]
```

GPU vocabulary

- Bell defines annotations used in LISA
- defines the common vocabulary, such as order and scope
- defines what an access or fence may carry

MP-cg-sys-plain.acq.litmus

```
 P0:cpu      | P1:gpu                ;
 MOV W0,#1   | r[acquire,sys] r0 y ;
 STR W0,[X1] | r[acquire,sys] r1 x ;
 MOV W2,#1   |                       ;
 STR W2,[X3] |                       ;
scopes: (sys (gpu (cta P1)))
exists (1:r0=1 /\ 1:r1=0)
```

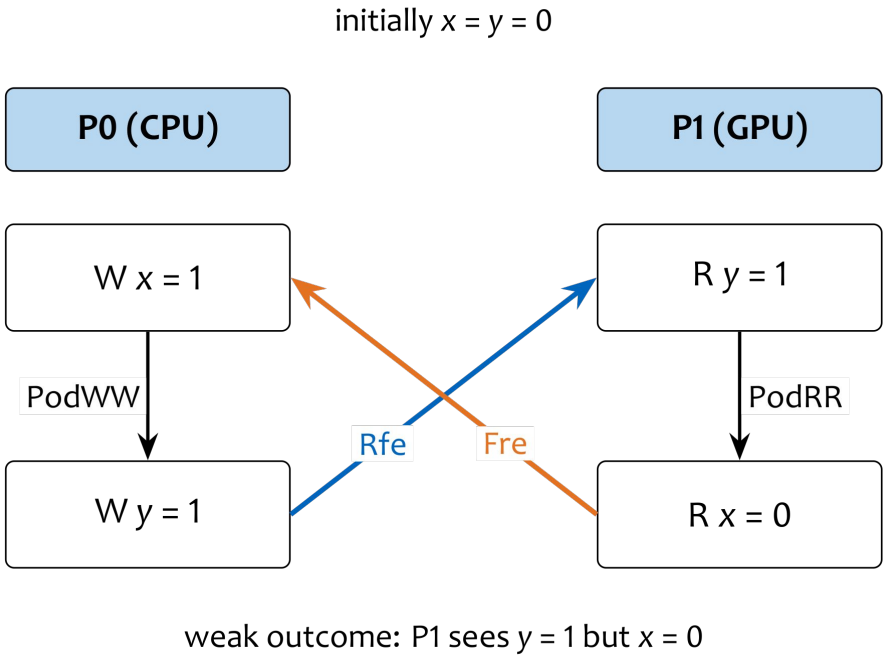
heterogeneous litmus format

- column header carries device (cpu or gpu)
- each token in column is translated to corresponding CUDA/HIP primitives

Concept: Relaxation Cycle

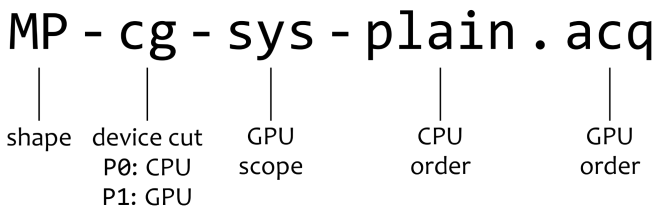
- PodWW Rfe PodRR Fre
- a test family is clear from this relaxation cycle

✔ independent of register or memory location names, ISA syntax
✔ each named family has a fixed cycle[4]
✔ compact



[4]J. Alglave, L. Maranget, M. Tautschnig, "Herding Cats: Modelling, Simulation, Testing, and Data Mining for Weak Memory", TOPLAS 36(2), 2014. https://doi.org/10.1145/2627752

Contribution: Cycle-based systematic enumeration



how test is named (excluding extension .litmus)

- Duplicates are removed
- CPU orders are varied only at sys scope
- **Adding a new shape is one line**
- Total **10,979** tests across **29** shapes

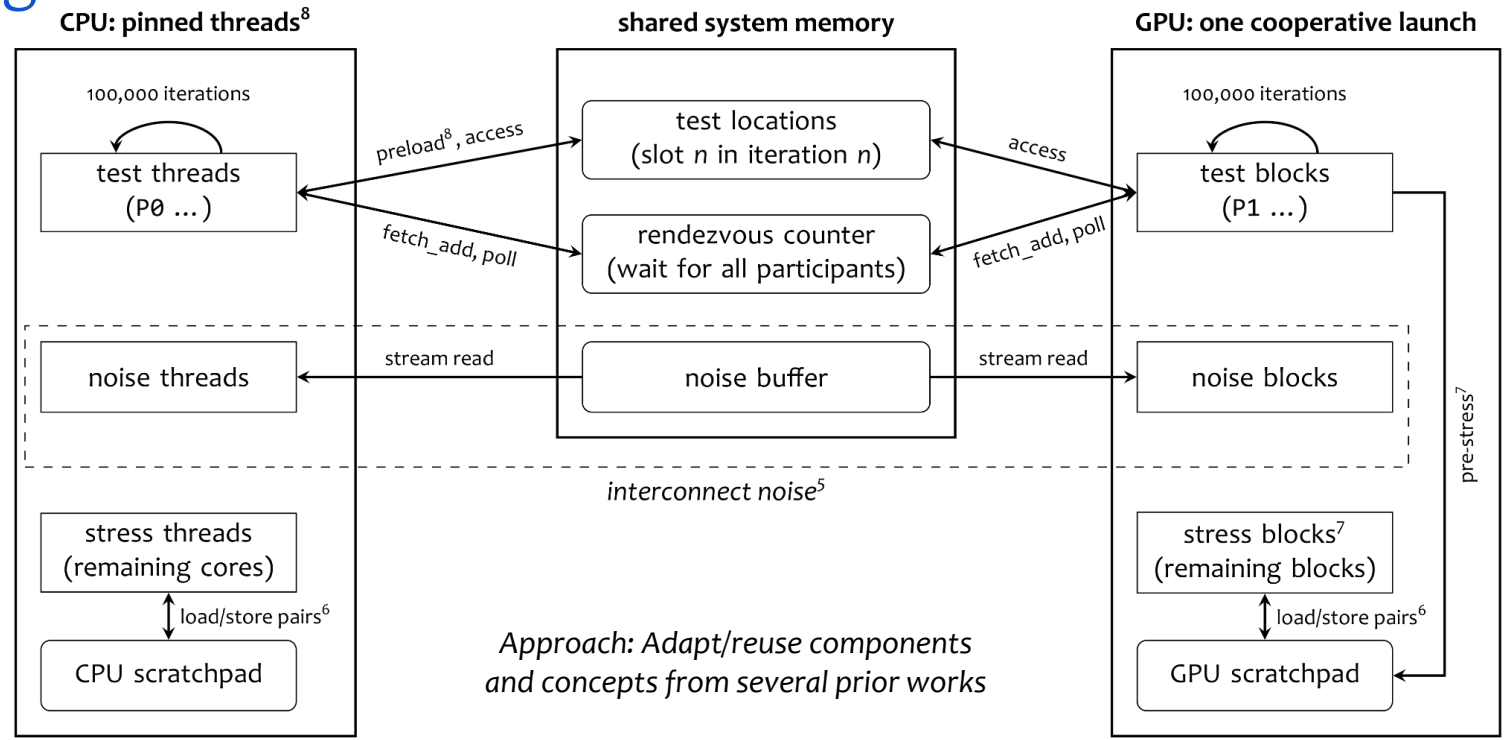
Axis	**Values**
Shape	MP, SB, LB, IRIW, WRC, …
Device cut	which threads run on the CPU and which on the GPU
GPU scope	cta, gpu, sys
CPU order	AArch64: plain, ra, sy, st, ld. x86_64: plain, mf
GPU order	rlx, acq, rel, ra, sc, fsc, facq, frel, fra

Other contributions

- Interconnect noise
  - both devices stream-read a shared buffer larger than LLC to load the link
  - adapted from Fusco et al.[5]
- Environment diagnostics
  - Shows what was requested vs. what actually ran (e.g., CPU stress) → fix before next run

[5] L. Fusco et al., "Understanding Data Movement in Tightly Coupled Heterogeneous Systems: A Case Study with the Grace Hopper Superchip", arXiv:2408.11556, 2024. https://arxiv.org/abs/2408.11556

Design



[5] L. Fusco et al., "Understanding Data Movement in Tightly Coupled Heterogeneous Systems: A Case Study with the Grace Hopper Superchip", arXiv:2408.11556, 2024. https://arxiv.org/abs/2408.11556

[6] T. Sorensen, A. F. Donaldson, "Exposing Errors Related to Weak Memory in GPU Applications", PLDI '16. https://doi.org/10.1145/2908080.2908114

[7] R. Levine, A. Goens, T. Sorensen, cuda-litmus. https://github.com/reeselevine/cuda-litmus

[8] L. Maranget, J. Alglave et al., herdtools7 (litmus7). https://github.com/herd/herdtools7

Evaluation

- rented NVIDIA DGX Spark instance, CUDA 12.9
- subset of 637 tests
- weak outcomes appear between 1 and 13,579 per 100,000
- 109 in unordered observed vs 0 fully fenced

Part	#Tests	#Observed
Unordered	135	109
Fully fenced	135	0
Full grid (6 shapes)	367	84
Total	637	193

Evaluation

GPU scope	GPU order `rlx`	`sc`	fence
MP-cg: CPU writes, GPU reads			
`cta`	●	○	●
`gpu`	●	○	○
`sys`	●	○	○
MP-gc: GPU writes, CPU reads			
`cta`	●	●	●
`gpu`	●	●	●
`sys`	●	○	○
SB-cg			
`cta`	●	●	●
`gpu`	●	●	●
`sys`	●	○	○
LB-cg			
`cta`	●	○	●
`gpu`	●	○	○
`sys`	●	○	○

● observed ○ not observed

fence column: same result for `fsc`, `frel` and `fra`

sys row: same result for all CPU orders (`plain`, `ra`, `st`, `ld`, `sy`)

- Six shapes run all combinations
  - MP, SB, LB, CoRR, CoWR and CoRW2
- No coherence test family observed
- GPU relaxed: observed every row, even full CPU fence
- GPU sc and fences: at cta/gpu still observed, but no sys
  - sys: only scope include CPU
- *Not observed ≠ forbidden*

Conclusion

- Generating heterogeneous litmus tests from relaxation cycles works end to end on hardware
- ISA-agnostic cycle + common GPU vocabulary layer → extensible to other kernel languages

Backup

```
#ifdef ACQ_REL
    cuda::memory_order store_order = cuda::memory_order_release;
    cuda::memory_order load_order = cuda::memory_order_acquire;
#elif defined(RELAXED)
    cuda::memory_order store_order = cuda::memory_order_relaxed;
    cuda::memory_order load_order = cuda::memory_order_relaxed;
#else
    // Default: release/acquire
    cuda::memory_order store_order = cuda::memory_order_release;
    cuda::memory_order load_order = cuda::memory_order_acquire;
#endif

//------------------------------------------------------------------------//

#ifdef THREAD_1_IS_CPU
    // Thread 1 (consumer): load y, load x
#ifdef ACQ_REL
    uint r0 = test_locations[y_addr].load(cuda::memory_order_acquire);
#elif defined(RELAXED)
    uint r0 = test_locations[y_addr].load(cuda::memory_order_relaxed);
#else
    uint r0 = test_locations[y_addr].load(cuda::memory_order_acquire);
#endif
```

```
#ifdef THREAD_0_IS_GPU
      // Het barrier: wait for CPU thread handling Thread 1
      het_spin(&het_barriers[wg_offset + id_0], HET_BARRIER_EXPECTED,
               kernel_params->barrier_spin_limit);

      // Thread 0 (producer): store x=1, store y=1
      test_locations[x_0].store(1, cuda::memory_order_relaxed);
      test_locations[y_0].store(1, store_order);
#endif

#ifdef THREAD_1_IS_GPU
      // Het barrier: wait for CPU thread handling Thread 0
      het_spin(&het_barriers[wg_offset + id_1], HET_BARRIER_EXPECTED,
               kernel_params->barrier_spin_limit);

      // Thread 1 (consumer): load y, load x
      uint r0 = test_locations[y_1].load(load_order);
      uint r1 = test_locations[x_1].load(cuda::memory_order_relaxed);

      cuda::atomic_thread_fence(cuda::memory_order_seq_cst);
      read_results[wg_offset + id_1].r0 = r0;
      read_results[wg_offset + id_1].r1 = r1;
#endif
```

```
N1 = W[X]
N2 = R[Y, r0]
N3 = W[Y]
N4 = R[X, r1]

po: N1 -> N2
po: N3 -> N4
fr: N2 -> N3
fr: N3 -> N4

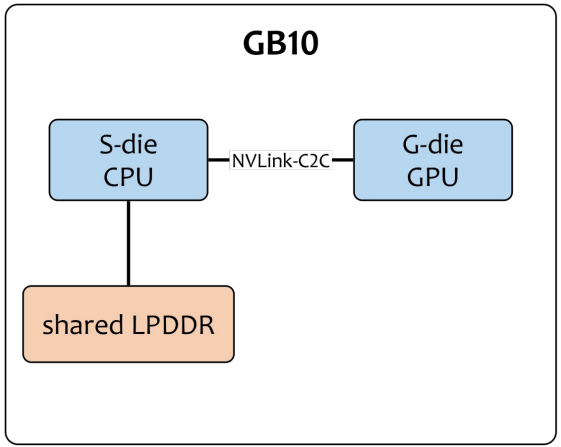
CPU_T1: N1; N2
CPU_T2: N3; N4
```

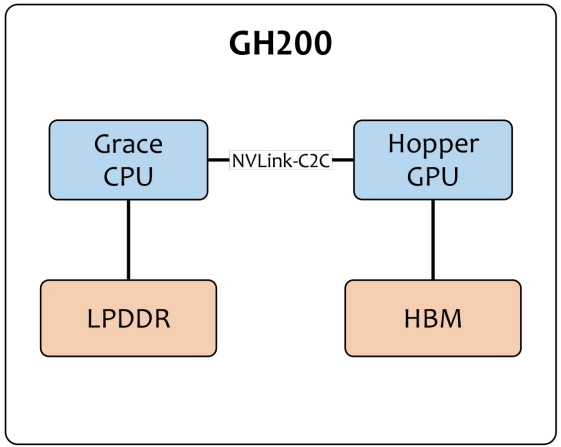
Listing 3.1: Execution graph file the Store Buffer litmus test.

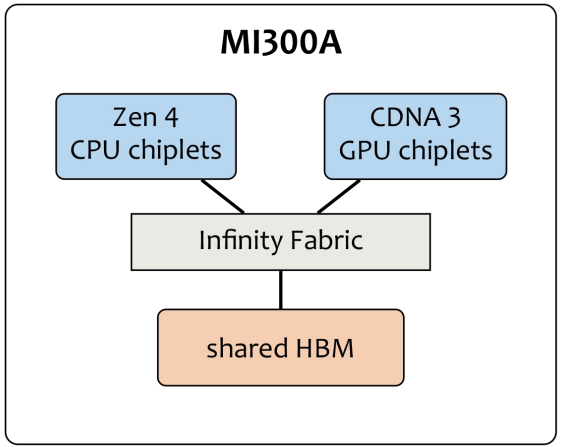
```
{
"configuration": {
    "type": "perp",
    "lang": "cuda",
    "cpu_obs": 1,
    "gpu_obs": 1,
    "mem_stress": false,
    "window": 5,
    "ws_pruned": true,
    "po_pruned": true,
    "prune_buffers": false,
    "optimized_heuristics": true,
    "mode": "HETEROGENEOUS",
    "mem_order": "relaxed"
},

"files": {
    "template": skeleton_perp_hetero.cu",
    "output_file": "sample.cu",
    "graph": "graphs/graph.txt",
    "cl_filename": "sample_cl.cl"
}
}
```

Listing 3.2: Sample .json configurations file for the compiler


GB10
S-die
CPU
NVLink-C2C
G-die
GPU
shared LPDDR


GH200
Grace
CPU
NVLink-C2C
Hopper
GPU
LPDDR
HBM


MI300A
Zen 4
CPU chiplets
CDNA 3
GPU chiplets
Infinity Fabric
shared HBM